

DySPAN 2026 Tutorial

Wireless Digital Twins

Yiming Li, Samuel Rivera and Tingjun Chen

Department of Electrical and Computer Engineering, Duke University

Overview of this tutorial

Goal: Provide a practical, hands-on guide to developing *scalable*, *physically-accurate*, and *high-fidelity* wireless digital twins (DTs)

Tools:

1. Publicly available geographic databases, including the OpenStreetMap (OSM) building footprints [REF] and USGS's nationwide lidar datasets [REF];
2. Open-source ray tracing (RT) and electromagnetic (EM) compute engines, (e.g., NVIDIA Sionna [REF]);
3. Real-world measurements that can be used to validate the performance of the developed digital twins [REF].

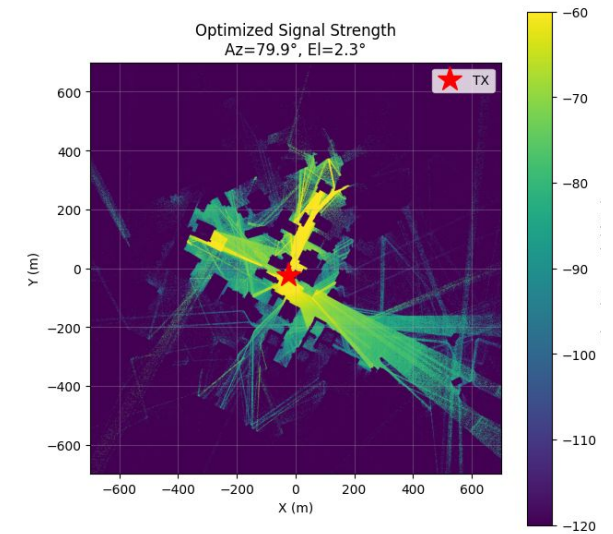
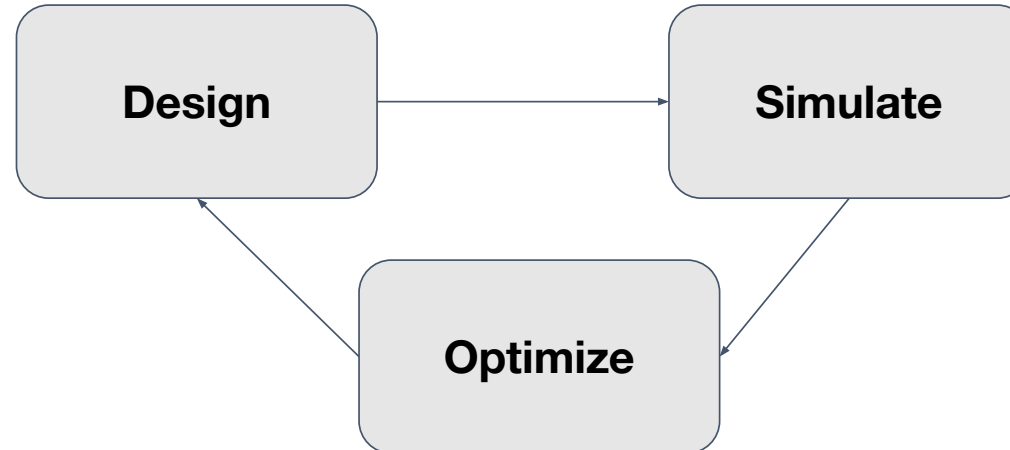
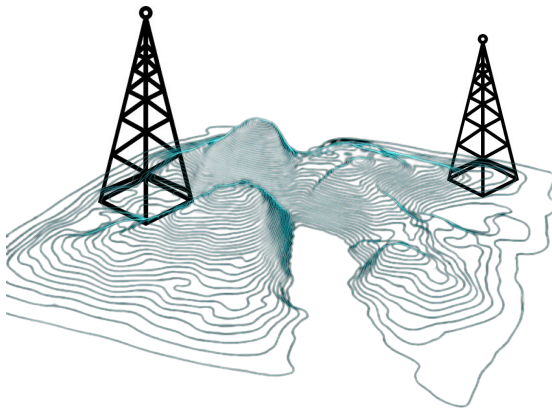
Format: Lecture + hands-on labs (Jupyter)

Section I

Introduction to Wireless Digital Twins

Overview

- Wireless Digital Twins model communication networks as virtual replicas embedded in their real-world physical environments.
- Having this representation provides opportunities to iteratively design, simulate and optimize networks without requiring real-world deployments.



Research Outlook

Physically accurate WDTs demand precise modeling of:

- Environment/scenario design
- Radio hardware
- Propagation Phenomena

Section II

Generating Scenes with Geo2SigMap

Blender-less Pipeline: Overview

Pure Python Pipeline for Efficient Digital Twins Generation

- Supports Sionna-RT (.xml)
- Scalable

```
$ scenegen bbox -71.0602 42.3512 -71.0484 42.3591 --data-dir scenes/Boston
```

```
[INFO] Check the bbox at http://bboxfinder.com/#42.3512,-71.0602,42.3591,-71.0484
[INFO] Using UTM Zone: EPSG:32619
[INFO]
[INFO] Ground Material Type:      Wet Ground      | Frequency Range: 1 - 10 (GHz)
[INFO] Building Rooftop Material Type: Metal      | Frequency Range: 1 - 100 (GHz)
[INFO] Building Wall Material Type: Concrete      | Frequency Range: 1 - 100 (GHz)
[INFO]
[INFO] Estimated ground polygon size: width=994m, height=901m
Parsing buildings: 100%|██████████| 389/389 [00:00<00:00, 1403.12it/s]
```

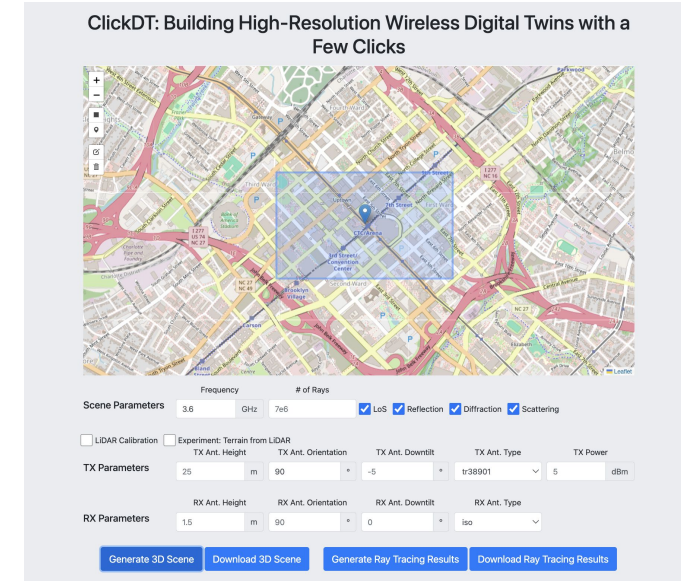
Various Interfaces

- CLI
- Web
- Python Package

CLI/Python Package

Tools & Framework Design

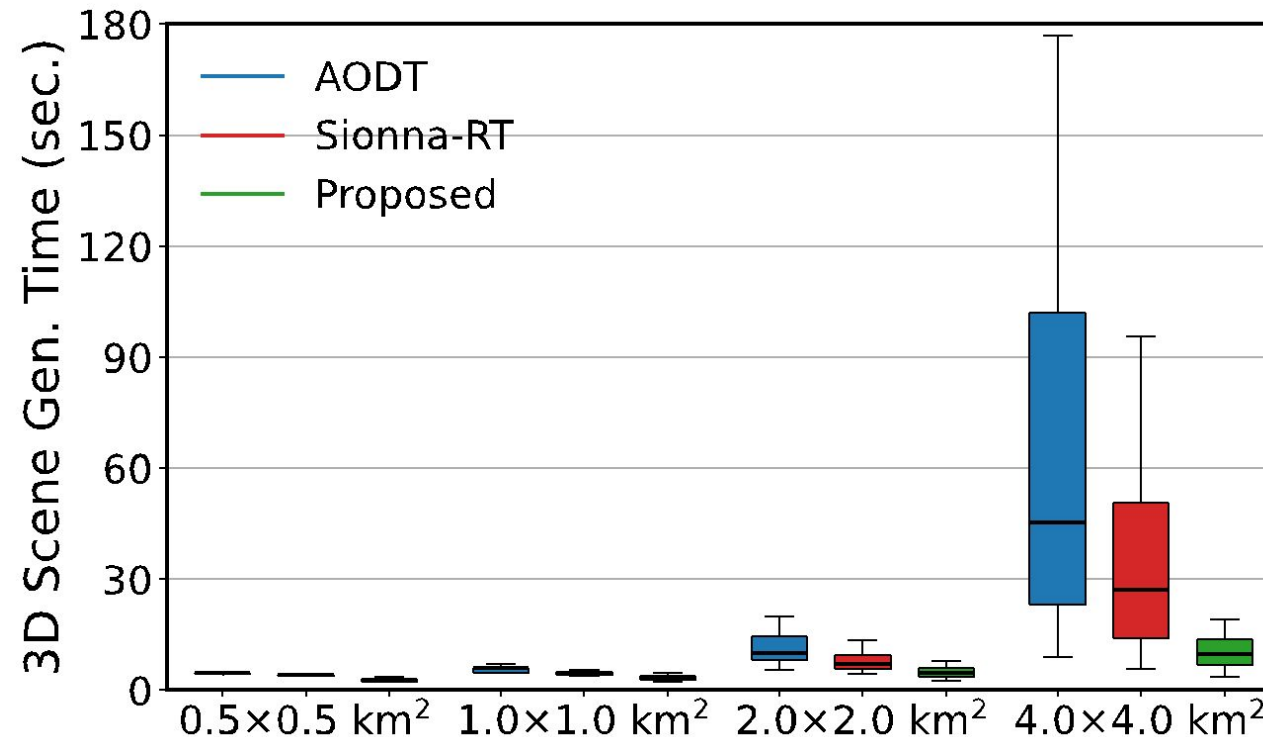
- OSMnx – OpenStreetMap data query/extract
- Open3D – Triangle Mesh/3D Object Operations
- triangle / SciPy (optional for BSD 3 license) – Delaunay Triangulation



Web

Blender-less Pipeline: Faster Generating Time

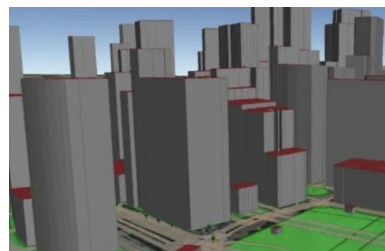
- Blender is too computationally expensive for “batched” scene generation.
- Our advantage: **pure Python pipeline optimized for faster generation.**



Distribution of 3D scene generation time across the 1,000 selected areas

Blender-less Pipeline: Overview

--- AODT Pipeline
... Sionna Pipeline
— Our Pipeline



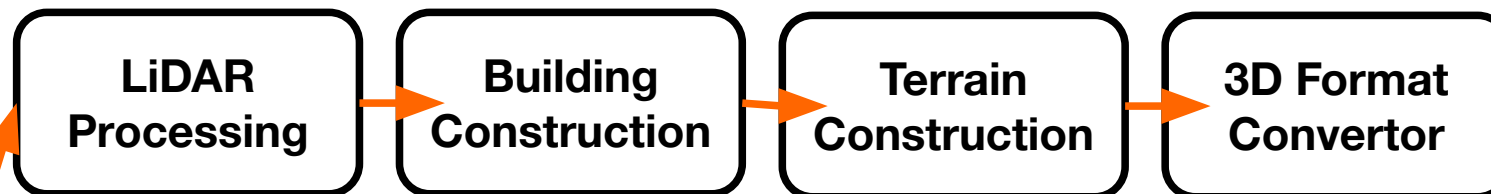
CityGML



OpenUSD Format



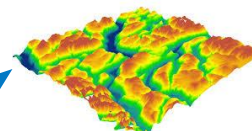
Geographical bounding box
[MinLat, MaxLat, MinLon,
MaxLon]



3DEP LiDAR dataset



OpenStreetMap (OSM)



SRTM DEM



Mitsuba3 Format

LiDAR Enhanced DT: Overview

High-Fidelity 3D Terrain and Building Meshes from LiDAR data

- Standard DEMs (Digital Elevation Models) often cap at **1m resolution**, limiting precision.
- **OpenStreetMap (OSM)** building data is frequently **outdated**, **incomplete**, or has non-standard values
 - (e.g., "3 floors" vs. "10m")

Our Approach

- Direct **LiDAR-based terrain mesh** generation with **controllable detail**.
- **Accurate 3D building reconstruction** with detailed rooftops and geometry ($\text{LOD} \geq 2$).

Data Source

- **USGS 3DEP** LiDAR dataset.

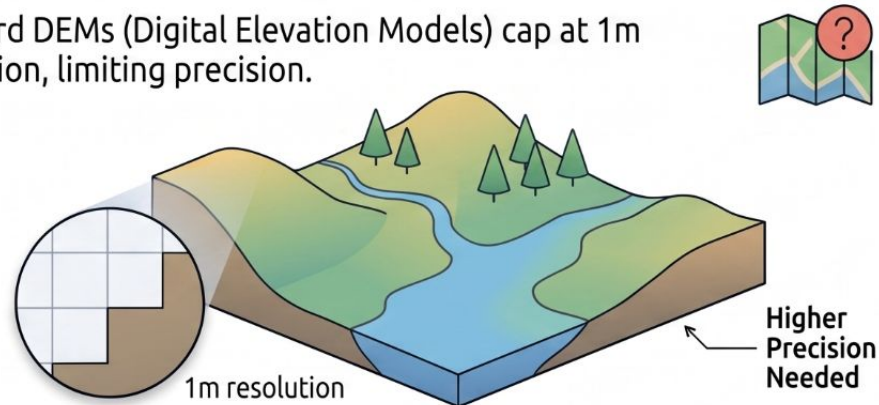
Tools & Framework

- **PDAL**, **Laspy** – preprocessing, filtering ground points.
- **PyVista** – mesh reconstruction.
- **City3D framework** – large-scale reconstruction from airborne LiDAR (open-source).

LiDAR Enhanced DT: Overview

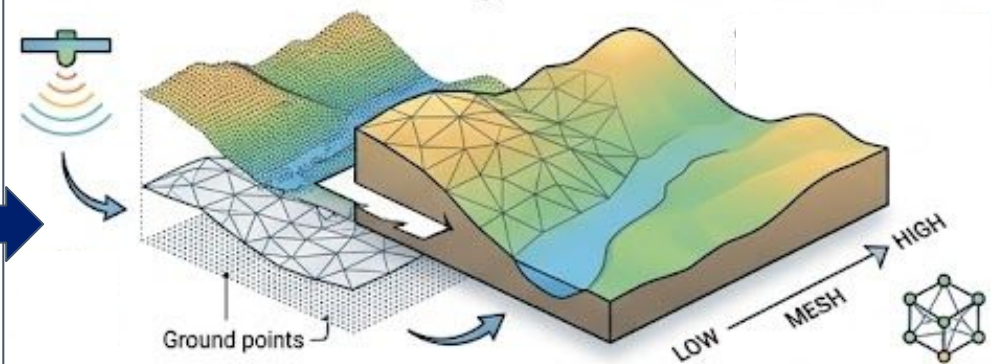
CURRENT LIMITATIONS - TERRAIN & RESOLUTION

Standard DEMs (Digital Elevation Models) cap at 1m resolution, limiting precision.



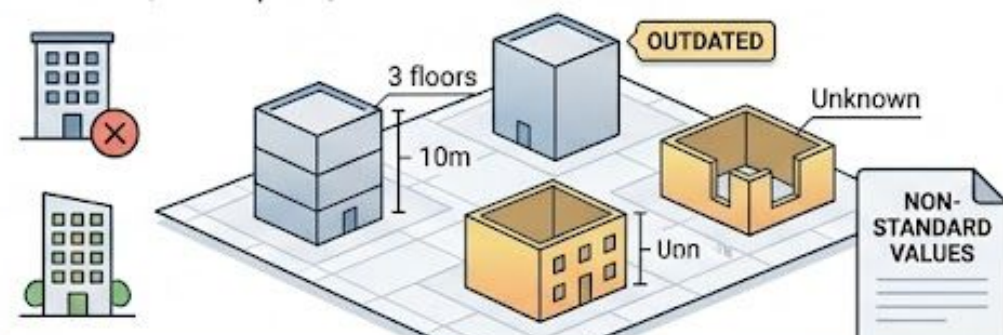
OUR APPROACH - HIGH-FIDELITY TERRAIN

Direct LiDAR-based terrain mesh generation with controllable detail



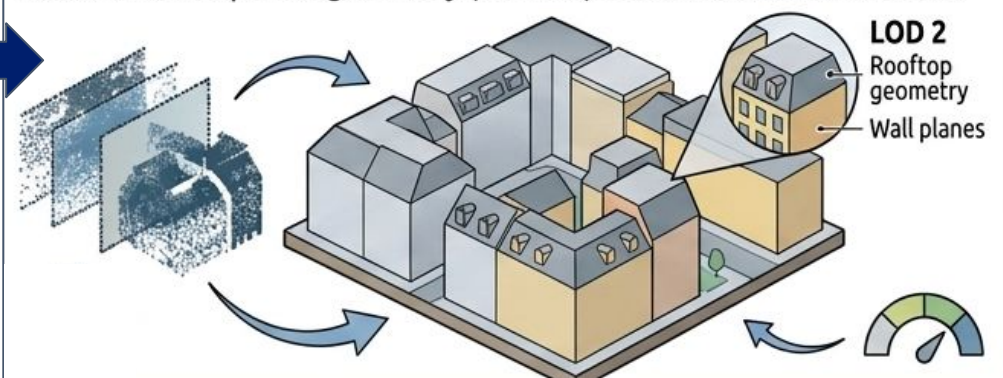
CURRENT LIMITATIONS - BUILDING DATA

OpenStreetMap (OSM) building data is frequently outdated, incomplete, or has non-standard values.



OUR APPROACH - ACCURATE BUILDING RECONSTRUCTION

Detailed rooftops and geometry (LOD ≥ 2) reconstructed from LiDAR

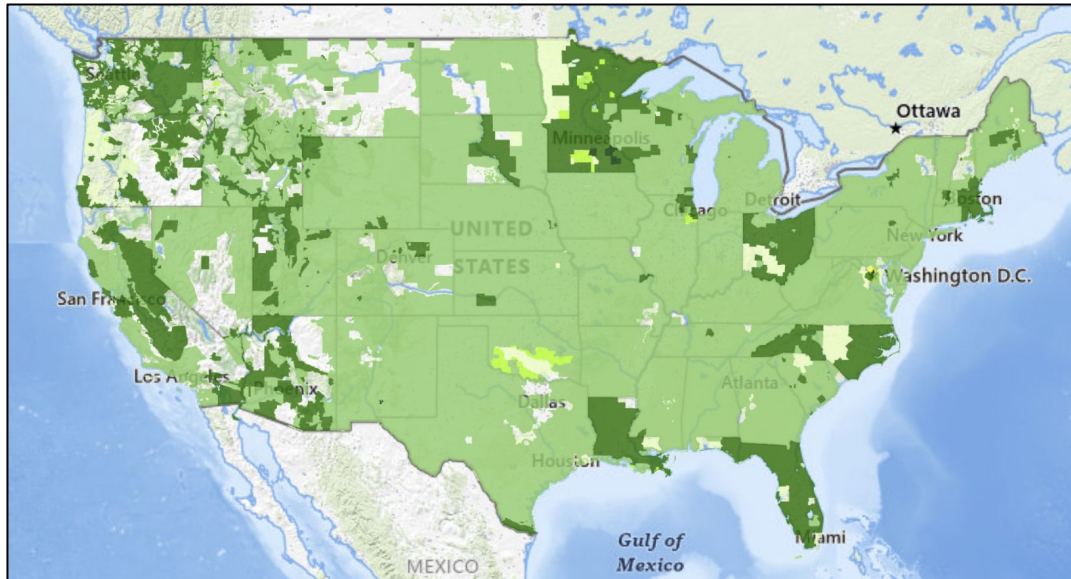


LiDAR Enhanced DT: USGS 3DEP LiDAR Dataset

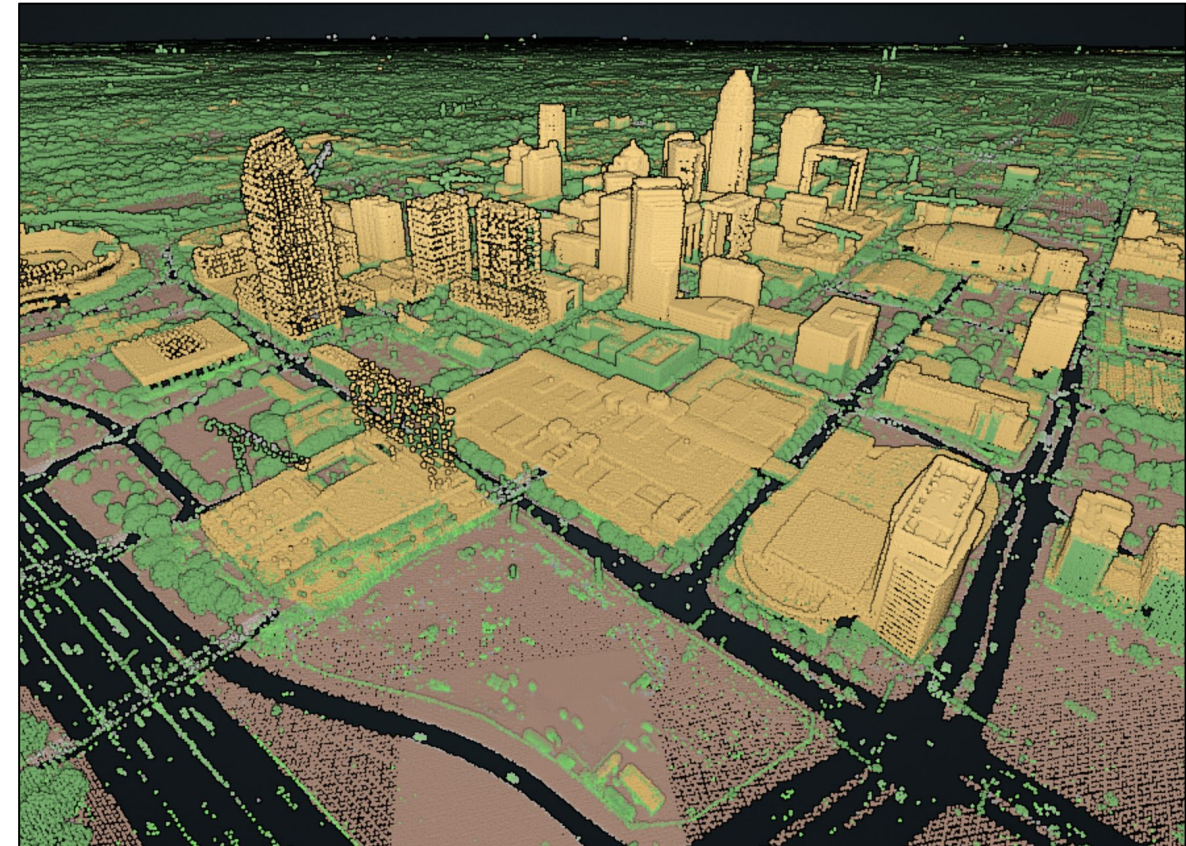
Nationwide publicly available lidar dataset for the continental United States.

Point Cloud Properties:

- Intensity
- Classification (Ground, Building, Foliage, ...)
- X, Y, Z coordinates



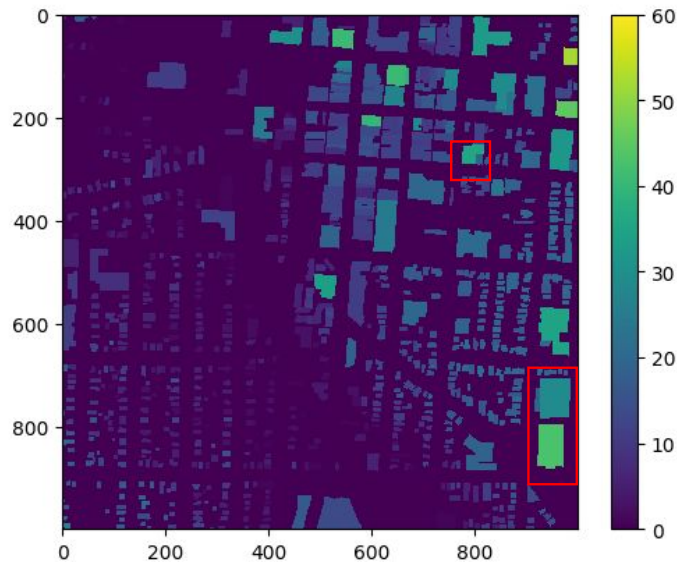
USGS LiDAR Data Availability Map



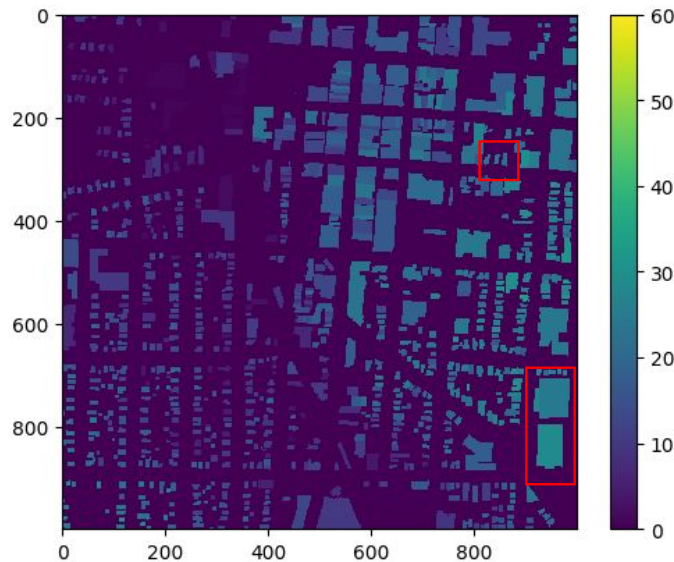
LiDAR Point Cloud
(Charlotte, NC)

Building Height Calibration

- Building height info is **not accurate** and is often **missing** in OpenStreetMap (OSM)
- Combine **2D building footprint** from OSM with **Z-value** from LiDAR



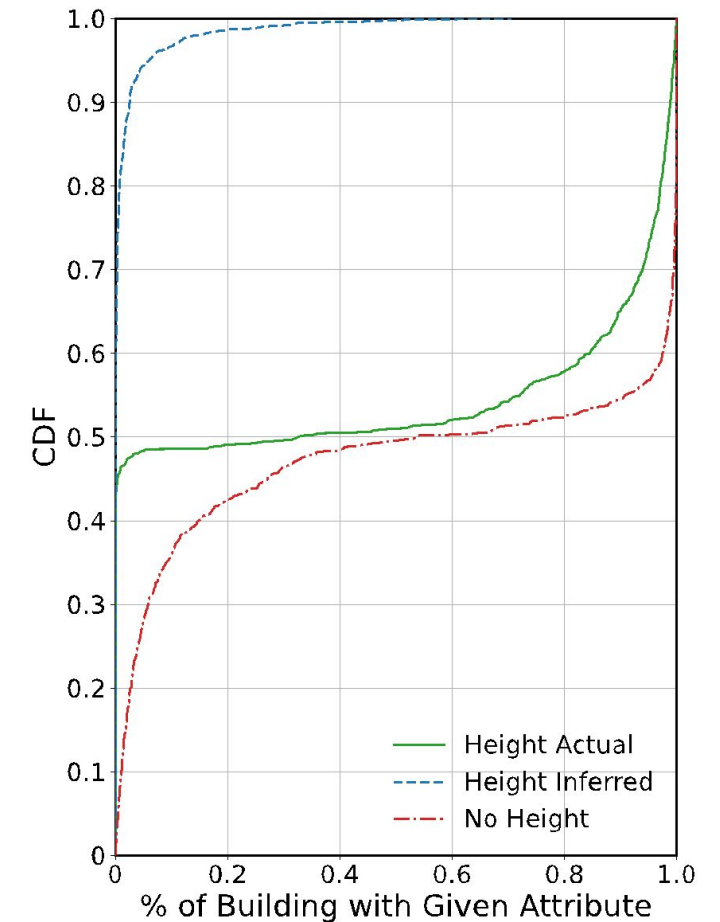
Building Height Map
(Calibrated by LiDAR data)



Building Height Map
(Raw OSM data)

Ann Arbor, MI

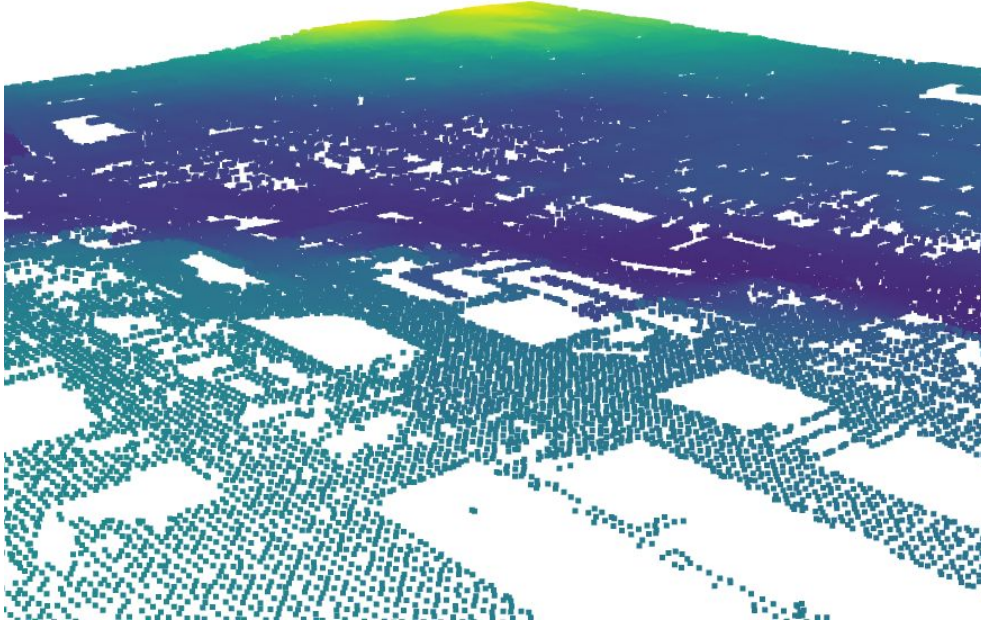
CDF of % of buildings with given attribute
across 1,000 selected 2 km×2 km area



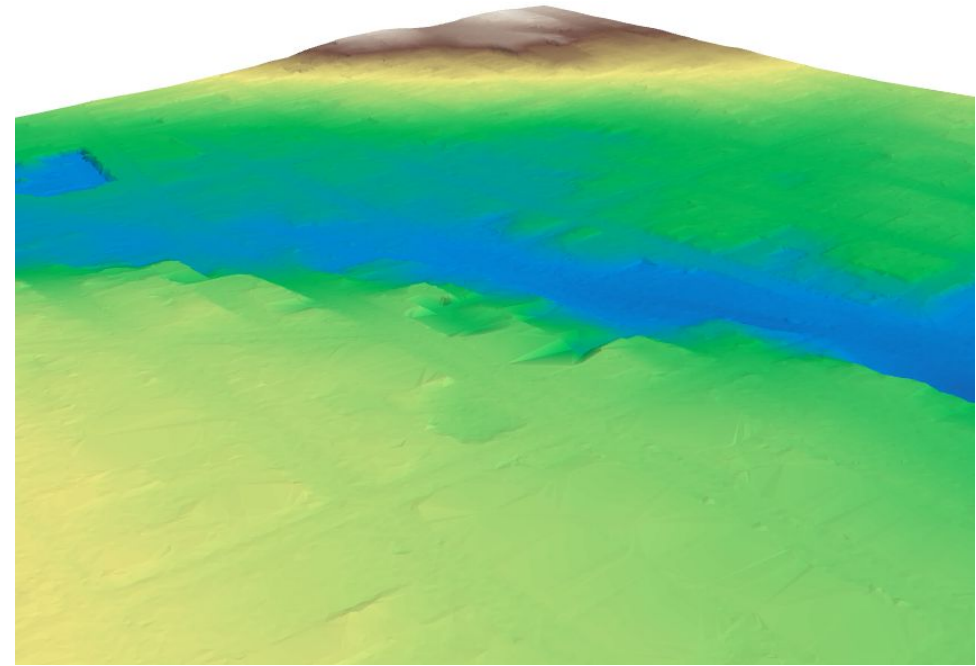
LiDAR Enhanced DT: High-fidelity Terrain Meshes

Direct **LiDAR-based terrain mesh** generation with **controllable detail**.

- **PDAL, Laspy** – preprocessing, filtering ground points.
- **PyVista** – mesh reconstruction.



LiDAR Point Cloud
(Points classified to **Ground**)



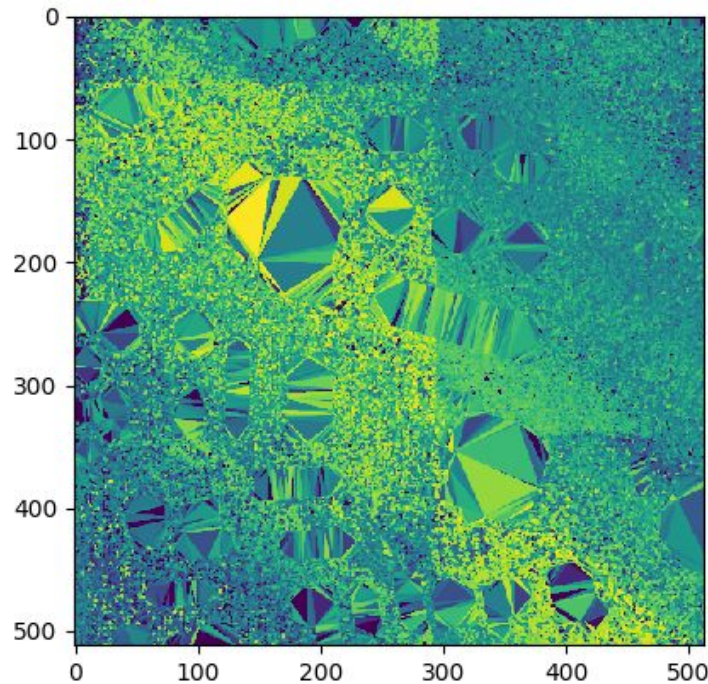
Formed Terrain Meshes

Ann Arbor, MI

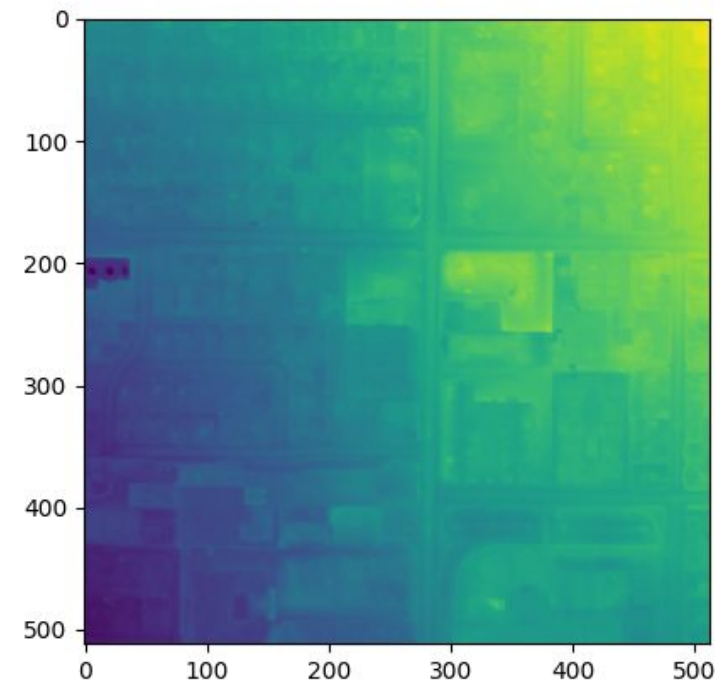
LiDAR Enhanced DT: High-fidelity Terrain Meshes

Direct **LiDAR-based terrain mesh** generation with **controllable detail**.

- USGS 3DEP DEM(Digital Elevation Model) terrain mesh.



Formed Terrain Meshes
(LiDAR Point Cloud)



Formed Terrain Meshes
(DEM)

Scene Generation: Single Corner API Example

```
# Replace '../scenes/Boston/scene.xml' with the
# path to your scene file
SCENE_FOLDER = "../../scenes/Dupont_flat"

# Configure the width/height of the scene
scene_width = 256.0
scene_height = 256.0

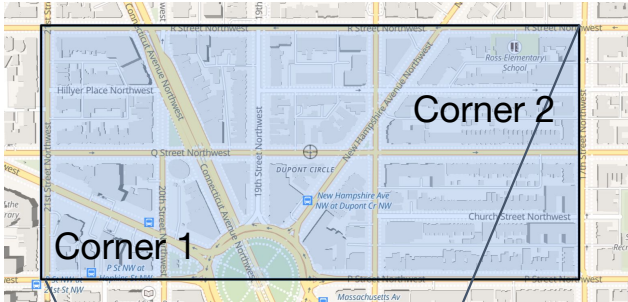
# Picking an initial lat/lon
# Using the sta
points = rect_from_point_and_size(
    lon=-77.046,
    lat=38.909,
    position="bottom-left",
    width=scene_width,
    height=scene_height
)
```

```
# Generate scene if it doesn't already exist
if not os.path.exists(f"{SCENE_FOLDER}/scene.xml"):
    # Generating the scene using the Python API
    print("Generating a flat scene for Dupont Circle")
    scene_gen = Scene()
    scene_gen(
        points,
        SCENE_FOLDER,
        None,
        osm_server_addr="https://overpass-api.de/api/interpreter",
        lidar_calibration=False,
        lidar_terrain=False,
        dem_terrain=False,
        generate_building_map=True,
        ground_material_type="mat-itu_concrete",
        rooftop_material_type = "mat-itu_metal",
        wall_material_type = "mat-itu_brick"
    )
```


Scene Generation: 4-Corner CLI Example

Command

Box Style



Save Dir



Optional Flags (v2.0.0)

```
$ scenegen bbox -77.046 38.909 -77.038 38.912 --data-dir scenes/Dupont_lidar --enable-lidar-terrain --enable-dem-terrain
```

[INFO] Check the bbox at <http://bboxfinder.com/#42.3485,-71.0547,42.3568,-71.0429>

[INFO] Using UTM Zone: EPSG:32619

[INFO]

[INFO] Ground Material Type:	Wet Ground		Frequency Range:	1	-	10	(GHz)
[INFO] Building Rooftop Material Type:	Metal		Frequency Range:	1	-	100	(GHz)
[INFO] Building Wall Material Type:	Concrete		Frequency Range:	1	-	100	(GHz)

[INFO]

[INFO] Estimated ground polygon size: width=997m, height=902m

Parsing buildings: 100%|████████████████████| 168/168 [00:00<00:00, 1383.61it/s]

Output Visualization

Find a way to insert/run visualization (?)

Section Tutorials

We now ask that you open and begin to work through the following tutorials:

- 0_geo2sigmap_scene_tutorial.ipynb

Section III

Site-specific Ray Tracing (RT) with Sionna-RT

Wireless Ray Tracing: The Basics

Wireless Channels are difficult to estimate in Complex Environments

Traditional models are **statistical** and environment-agnostic. This offers limited accuracy for:

- Urban Outdoor Settings
- High-frequency Systems (mmWave, 5G/6G)

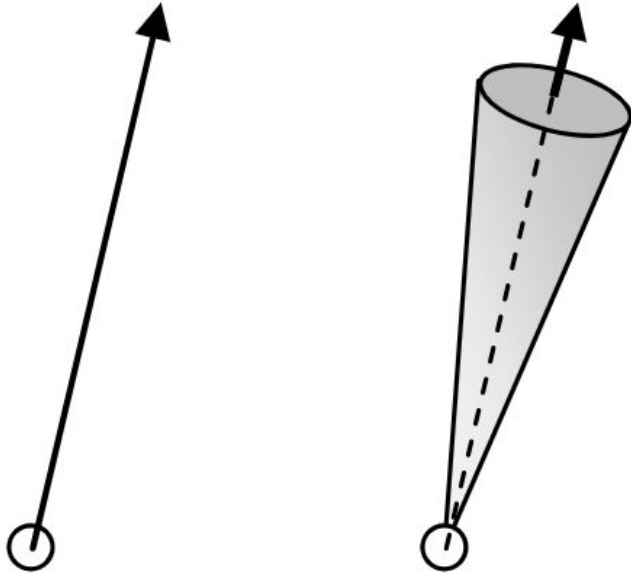
Ray tracing provides *site-specific* predictions:

- Limits the amount of measurement data required for downstream tasks
- Channel estimates are **deterministic** in nature.



Wireless Ray Tracing: The Basics

How do they work?



- Transmitters are modeled as **point** sources.
- Wavefronts are represented using lines (rays).
- Each **ray** can be **traced** through the scene to create a series of **paths**.

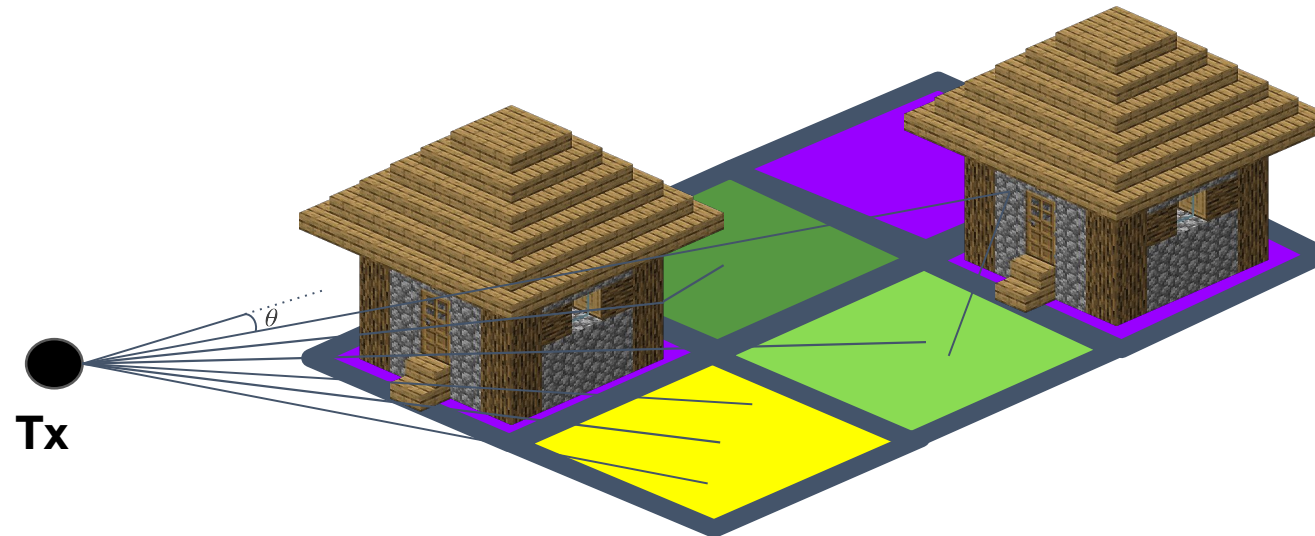
Each ray carries energy!

- The energy (cross-section) is often modeled as an expanding “**tube**”.
- Channel estimates are discretized in space using:
 - 2-D Grids: **Maps**
 - Points: **Receivers**

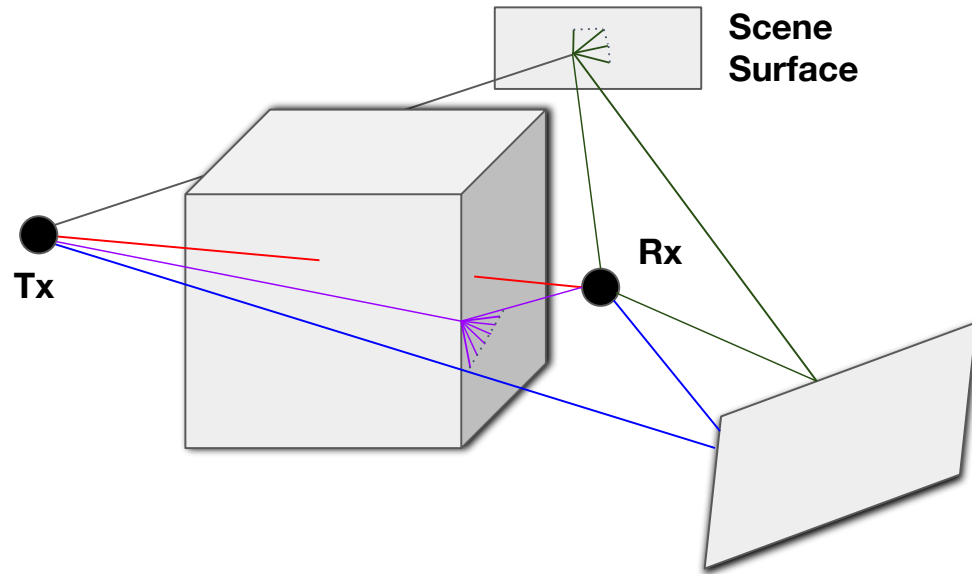
Wireless Ray Tracing: The Basics

Point to Grid Example

- Rays are **launched uniformly** from Tx.
 - This is formally known as the **Shooting and Bouncing of Rays (SBR)**
- Measurements are discretized into equal sized pixels on a fixed 2-D plane.
- Measurements (visually) represent a spatial average of signal energy.
- Commonly known as a Radio or **Coverage Map**.



Wireless Ray Tracing: The Basics



Point to Point Example

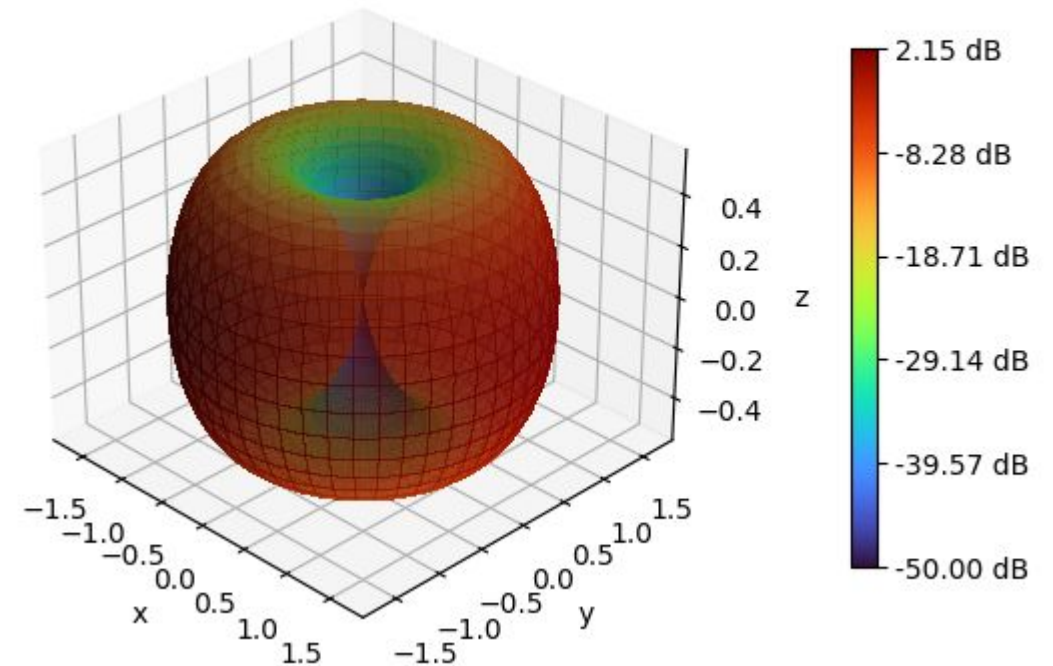
- Rays are **launched** from Tx using **SBR**.
- Connecting paths are isolated using the **image method**.
 - Virtual sources are used to validate Tx–Rx connections.
- Colored rays denote path types:
 - — Refraction
 - — Specular Reflection
 - — Diffuse Reflection (Scattering)
 - — Diffraction

Wireless Ray Tracing: The Basics

Other Considerations

- Alongside freespace and multipath losses, each endpoint has an associated **radiation pattern**.
- Every propagation interaction must consider **material characteristics** for frequency-accurate estimates:
 - Relative Permittivity
 - Relative Conductivity
- Ray tracing is computationally intensive—hardware accelerated solutions should balance the tradeoff between **speed** and **accuracy**.

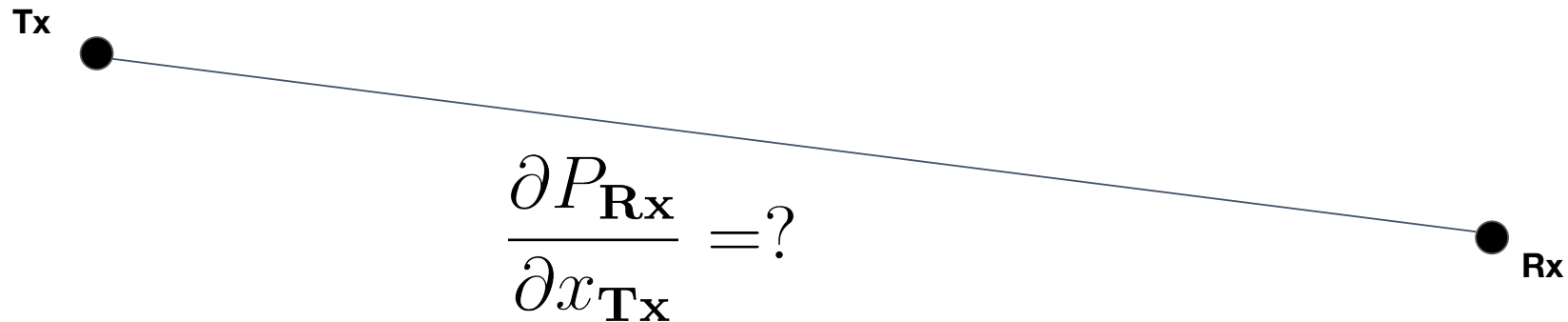
3D visualization of the radiation pattern $G(\theta, \varphi)$



Why Sionna RT?

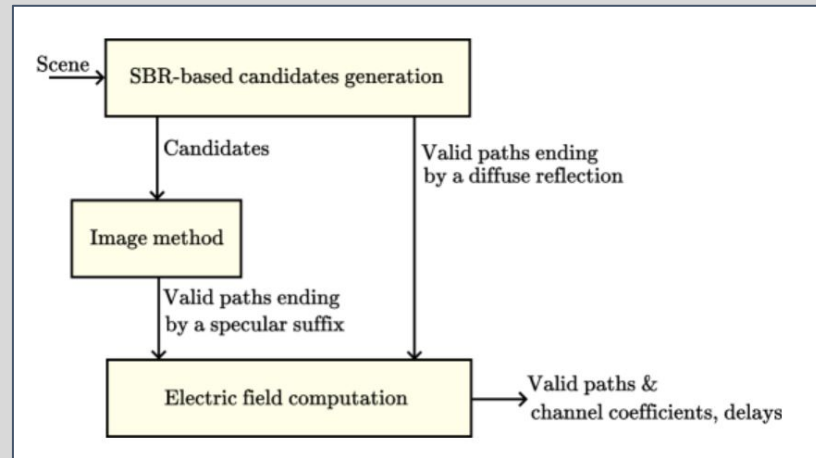
Sionna RT introduces several key advantages over proprietary ray-tracing tools

- Hardware accelerated: CUDA-enabled execution supports large-scale ray-tracing workloads at high speed
- Fully differentiable: Built on Mitsuba 3, enabling gradient-based optimization/ML through the ray-tracing pipeline
- Open source: Publicly available on GitHub, with contributions from the broader research and developer community



Working with Sionna RT: Solver Classes

PathSolver(): Point to Point



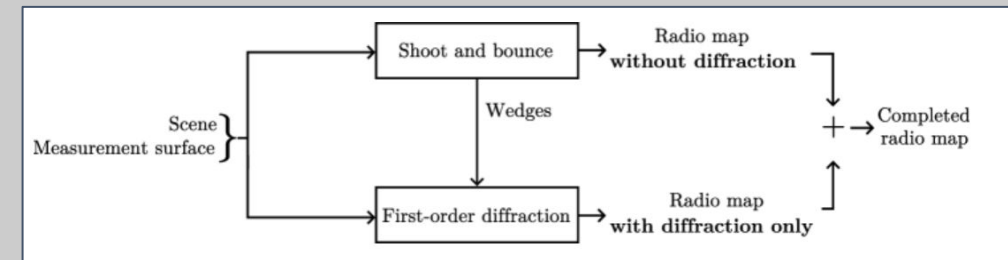
Input: "Scene"

Mesh
Transmitters
Receivers
Radiation Patterns

Output

Valid Paths
Channel Coefficients
Delays

RadioMapSolver(): Point to Grid



Input: "Scene"

Mesh
Transmitters
Map Definition(s)
Radiation Patterns

Output

Received Signal Strength (RSS)
Signal to Interference plus
Noise Ratio (SINR)

Code Skeleton: RadioMapSolver()

```
SCENE_FOLDER = "../..../scenes/Dupont_flat"
scene = load_scene(f"{SCENE_FOLDER}/scene.xml")

# Set the operating frequency in Hz (here, 3.65 GHz
corresponds to the CBRs band)
scene.frequency = 3.65e9

# Create a Planar Array with 1 element
scene.tx_array = PlanarArray(
    num_rows=1,
    num_cols=1,
    vertical_spacing=0.5,
    horizontal_spacing=0.5,
    pattern="iso",          # isotropic pattern
    polarization="V"       # vertical polarization
)
```

```
# Create a single transmitter (tx0)
tx0 = Transmitter(
    name="tx",
    position=[-5, 5, 120]
)

# Add the transmitter instance to the scene
scene.add(tx0)

rm_solver = RadioMapSolver()

rm = rm_solver(scene,
    max_depth=5,
    los=True,
    refraction=False,
    cell_size=(1., 1.),
    precoding_vec=None,
    samples_per_tx=int(1e8)
)
```

Code Skeleton: PathSolver()

```
# Define Receive Array (single dipole element)
scene.rx_array = PlanarArray(
    num_rows=1,
    num_cols=1,
    vertical_spacing=0.5,
    horizontal_spacing=0.5,
    pattern="dipole",
    polarization="V"
)
# Create and add a Receiver to the Scene
rx = Receiver(
    name="rx",
    position=[0, 0, 1.5],
    orientation=[0, 0, 0]
)

scene.add(rx)
```

```
# Create an instance of the PathSolver Class
solver = PathSolver()

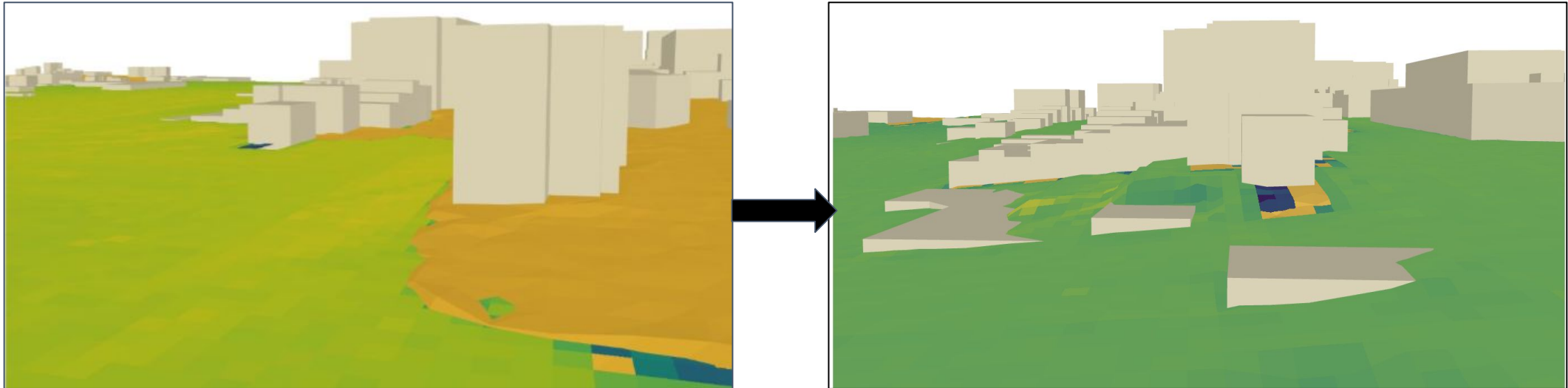
# Generate Paths object
paths = solver(scene, max_depth=3,
    los=True,
    specular_reflection=True,
    samples_per_src=int(1e8)
)

# Preview Scene
scene.preview(paths=paths);
```

Advanced Use Cases

Sionna-RT v2.0.0 supports **irregular measurement planes**

- Instead of using a 2-D grid for channel measurements, you can also characterize signal using the lidar terrain mesh triangles.
- Our notebook example, `1_sionna_coverage_map.ipynb`, adapts this feature to quantify coverage along the rough terrain.



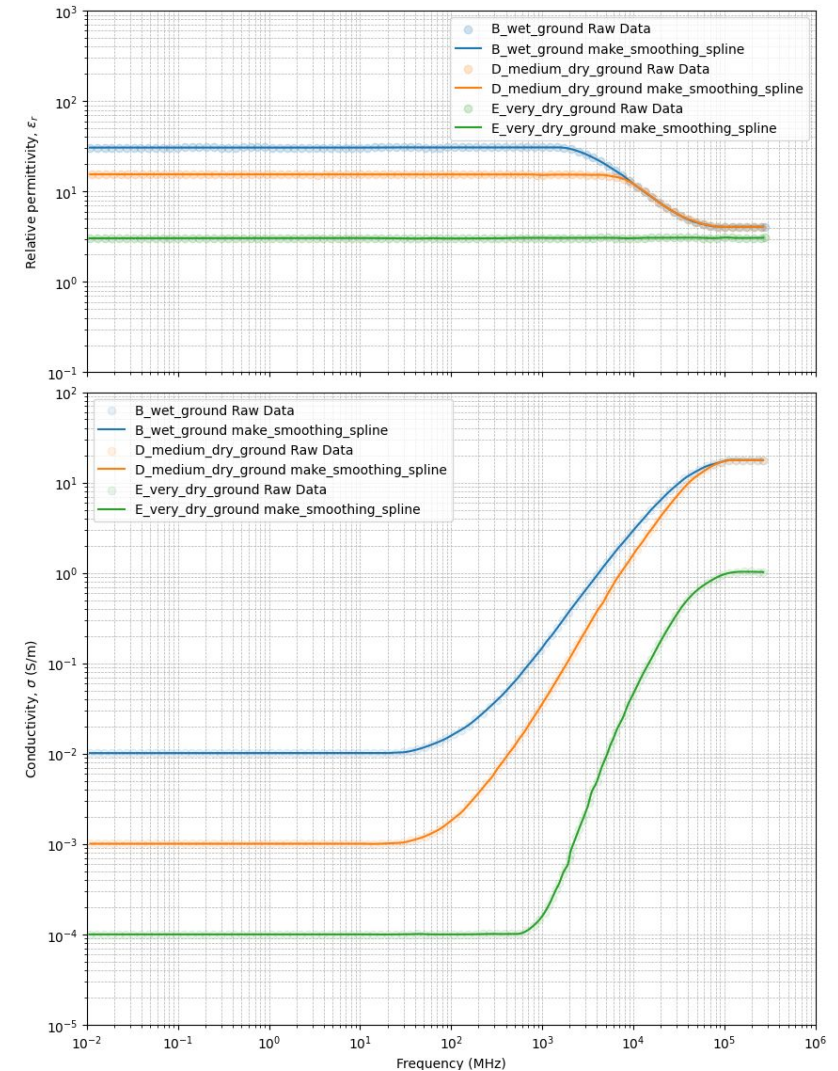
Advanced Use Cases

Ground material definitions can be characterized for **extended frequency ranges**.

- Out of the box, Sionna RT uses ITU P.2040-3
 - Supports simulations using **1-10 GHz**
- Our notebook in Section IV, `5_scene_nist_28ghz_boulder.ipynb`, extends support by using ITU-R P.527-3.
 - Supports simulations using **1 MHz - 100 GHz**

Sionna RT supports **mobile scenarios**

- Doppler shifts can be modeled over both short/long time horizons.
- Useful for generating time-varying CIRs.



Section Tutorials

We now ask that you open and begin to work through the following tutorials:

- 1_sionna_coverage_map.ipynb
- 2_sionna_rays_analysis.ipynb

Section IV

Validation with Real-World Measurements

Validation

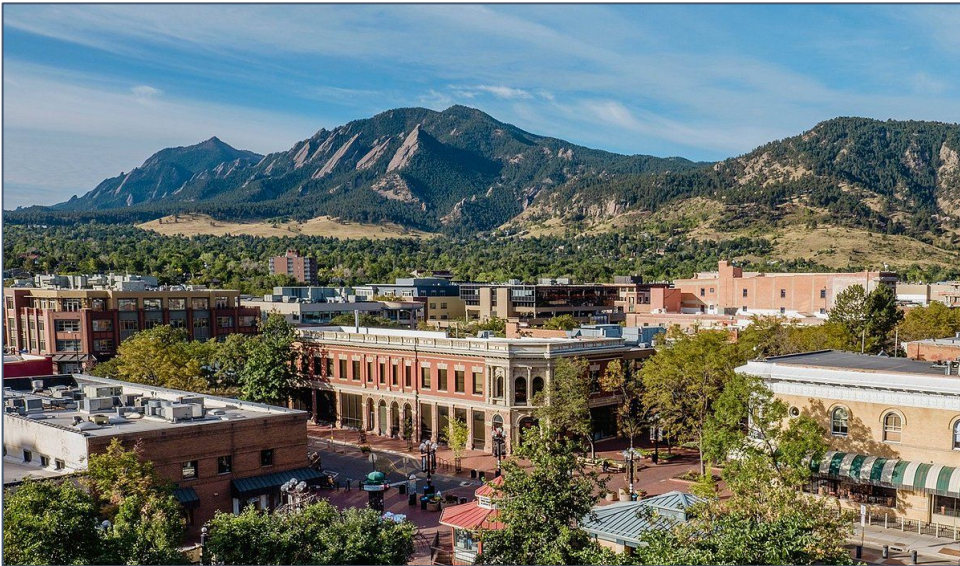
With ray tracing data widely accessible, how accurate are these results?

- Beyond site-specific analysis, WDTs are primarily targeted for generating large scale synthetic datasets.
- Real world measurements have now become key for fine-tuning simulation designs.
- In this section of the tutorial, we aim to assess the accuracy of two WDTs:
 - Sionna RT
 - Geo2SigMap's DL Model (U-Net)
- Our notebook examples quantify differences in metrics including:
 - RSRP, Coverage, etc.
 - Channel Gain, AoA, AoD and Path Delay.

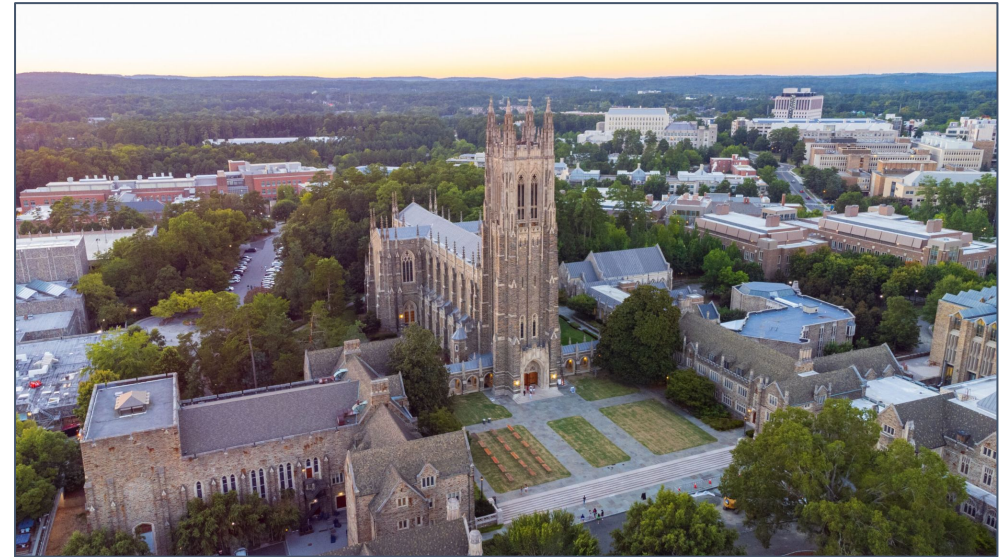
Real World Datasets

In this section of the tutorial, we aim to assess the accuracy WDTs using two real world measurement campaigns:

Boulder, Colorado: 28 GHz



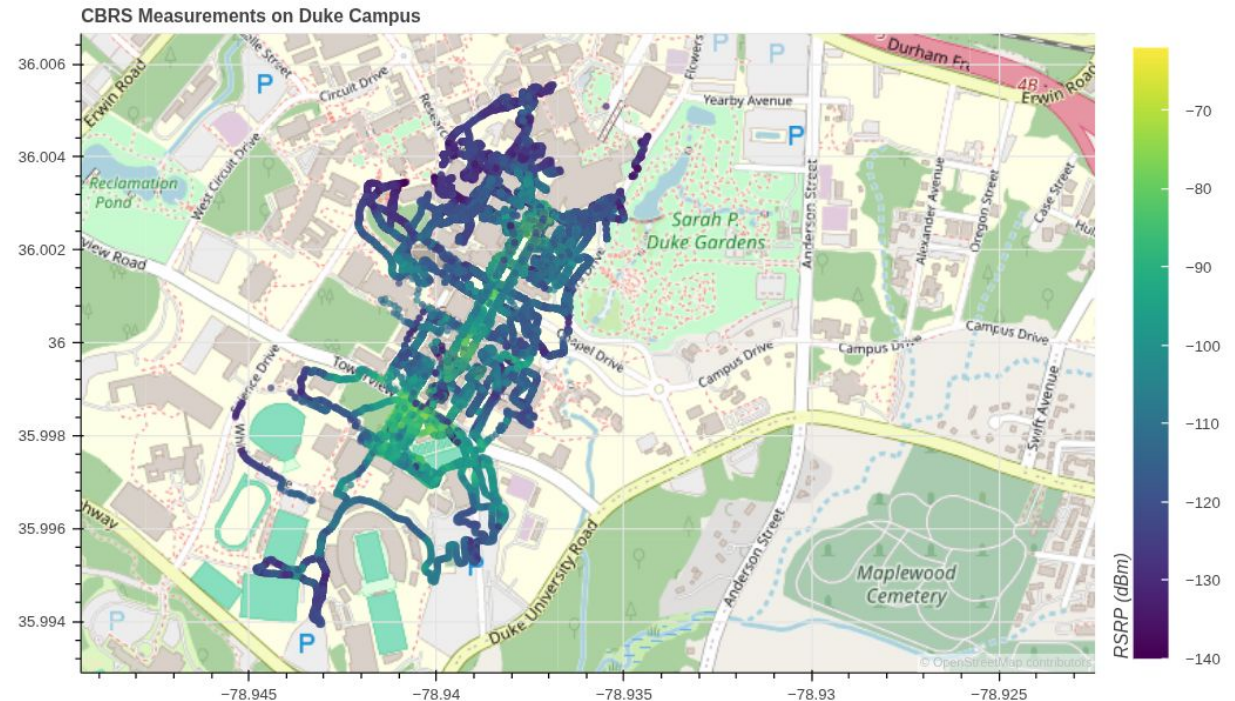
Durham, NC (Duke University): 3.55-3.7 GHz



Duke University: CBRS Measurement Campaign

Overview

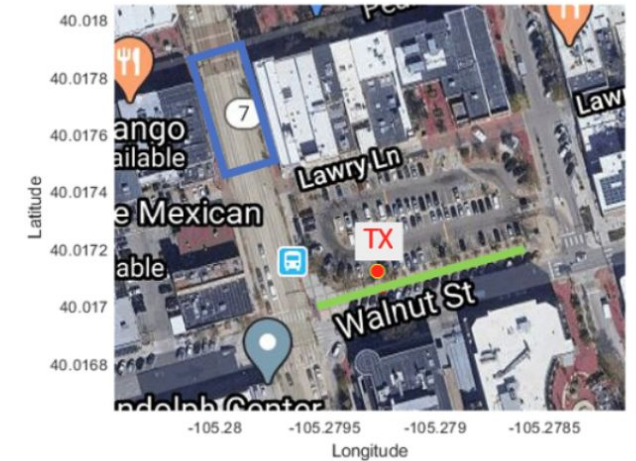
- Captured between 07/2023 - 11/2023.
- 6 LTE cells operating in CBRS band (3.55 - 3.7 GHz)
- Collected using 3 UE:
 - Samsung Galaxy A42
 - Google Pixel 5
 - RPi 4B w/ LTE HAT
- Used to validate Geo2SigMap (DySPAN 2024)



Boulder, CO: NIST Dataset

Overview

- Captured in the Summer of 2025
- Offered through NIST's *NextG Channel Measurement and Modeling* program.
- 28 GHz "Channel Sounder" is placed in downtown Boulder, using a wheeled cart to capture channel measurements.
- Environment/transmission characteristics are defined for adaptations to ray tracing



LOS

NLOS

Comparison Considerations

Section Tutorials

We now ask that you open and begin to work through the following tutorials:

1. `3_visualize_measurements.ipynb`
2. `4_ml_coverage_map.ipynb`
3. `5_scene_nist_28ghz_boulder.ipynb`